МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №4

з дисципліни **"Функціональне програмування"**

на тему **"РЕКУРСІЯ ТА МЕМОІЗАЦІЯ"**

Варіант 14. Обходи n-арного дерева

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: засвоїти побудову рекурсивних алгоритмів через базовий випадок і рекурсивний крок, зрозуміти обмеження стеку викликів у CPython та відсутність оптимізації хвостових викликів, навчитися усувати повторні підзадачі за допомогою `functools.lru_cache`; реалізувати рекурсивні обходи, пошук і висоту n-арного дерева (варіант 14), а також quicksort, обходи бінарного дерева та мемоізований Фібоначчі з методички.

Теоретичні відомості:

Рекурсія зводить задачу до однієї або кількох менших задач того самого вигляду. Щоб рекурсія завершилась, потрібен базовий випадок — вхід, для якого відповідь відома без подальших викликів (наприклад, листок дерева), і рекурсивний крок, який гарантовано наближає задачу до бази (перехід до дітей вузла). Кожен виклик у CPython займає кадр стеку, глибина якого обмежена значенням `sys.getrecursionlimit()` (типово 1000); перевищення дає `RecursionError`. Оптимізація хвостових викликів відсутня за задумом розробників мови, тому для даних великої глибини замість рекурсії застосовують ітерацію з явним стеком або чергою.

Мемоізація — збереження вже обчислених результатів чистої функції для повторного використання. Якщо рекурсивний алгоритм багато разів розв'язує однакові підзадачі (Фібоначчі, висота спільного піддерева, що зустрічається у дереві кілька разів), декоратор `@lru_cache` перетворює експоненційну кількість викликів на лінійну. Методи `cache_info()` і `cache_clear()` дають змогу спостерігати за кешем і скидати його у тестах. Для кешування аргументи мають бути хешованими, тому незмінний вузол дерева з `tuple` дітей підходить ідеально.

Хід роботи:

1. За варіантом 14 у `ntree.py` описала незмінний вузол n-арного дерева `TreeNode` (frozen dataclass зі значенням і `tuple` дітей; `eq=False` дає хеш за ідентичністю, тому вузол можна передавати в `lru_cache` без рекурсивного хешування піддерева), рекурсивні генератори обходів `preorder` і `postorder` (база — листок,

крок — `yield from` для кожної дитини) та рекурсивний пошук `find_path`, який повертає шлях значень від кореня до знайденого вузла або `None`:

```python
@dataclass(frozen=True, slots=True, eq=False)
class TreeNode:
    """Вузол n-арного дерева: значення і незмінний tuple дітей.

    eq=False → хеш за ідентичністю, тому вузол можна передавати в lru_cache
    без рекурсивного хешування всього піддерева.
    """

    value: int
    children: tuple[TreeNode, ...] = ()


def preorder(node: TreeNode) -> Iterator[int]:
    """Корінь → діти зліва направо (рекурсивний генератор)."""
    yield node.value
    for child in node.children:
        yield from preorder(child)


def postorder(node: TreeNode) -> Iterator[int]:
    """Діти зліва направо → корінь."""
    for child in node.children:
        yield from postorder(child)
    yield node.value


def find_path(node: TreeNode, target: int) -> tuple[int, ...] | None:
    """Рекурсивний пошук значення; повертає шлях значень від кореня або None.

    База: значення у поточному вузлі → шлях з одного елемента.
    Крок: перша дитина, у піддереві якої знайдено target.
    """
    if node.value == target:
        return (node.value,)
    for child in node.children:
        sub = find_path(child, target)
        if sub is not None:
            return (node.value, *sub)
    return None


def contains(node: TreeNode, target: int) -> bool:
    """Чи є значення в дереві (через рекурсивний find_path)."""
    return find_path(node, target) is not None

```

2. Реалізувала підрахунок вузлів і дві версії висоти дерева: `height_naive` без кешу та `height` з `@lru_cache`, яка кожне фізичне піддерево обчислює один раз:

```python
def count_nodes(node: TreeNode) -> int:
    """Кількість вузлів: 1 + сума по дітях."""
    return 1 + sum(count_nodes(c) for c in node.children)


def height_naive(node: TreeNode) -> int:
    """Висота дерева рекурсивно без кешу: листок → 1."""
    return 1 + max((height_naive(c) for c in node.children), default=0)


@lru_cache(maxsize=100_000)
def height(node: TreeNode) -> int:
    """Висота з мемоізацією піддерев: спільні піддерева рахуються один раз."""
    return 1 + max((height(c) for c in node.children), default=0)

```

3. Додала ітеративні альтернативи, які не залежать від глибини рекурсії: `dfs_iter` із власним стеком (дітей кладу у зворотному порядку, щоб зберегти порядок preorder), `bfs_iter` через `deque` та `height_iter` по рівнях; функція

`shared_tree` будує дерево, в якому всі діти вузла — одне й те саме піддерево, тож логічно воно має `2**depth` листків, а фізично лише `depth + 1` вузлів:

```python
def dfs_iter(node: TreeNode) -> list[int]:
    """Ітеративний preorder із власним стеком (дітей кладемо у зворотному порядку)."""
    stack, out = [node], []
    while stack:
        n = stack.pop()
        out.append(n.value)
        stack.extend(reversed(n.children))
    return out


def bfs_iter(node: TreeNode) -> list[int]:
    """Ітеративний обхід у ширину через deque — не залежить від глибини стеку."""
    queue, out = deque([node]), []
    while queue:
        n = queue.popleft()
        out.append(n.value)
        queue.extend(n.children)
    return out


def height_iter(node: TreeNode) -> int:
    """Висота ітеративно: рівень за рівнем через список поточного рівня."""
    level, depth = [node], 0
    while level:
        depth += 1
        level = [c for n in level for c in n.children]
    return depth


def shared_tree(depth: int, arity: int = 2) -> TreeNode:
    """Дерево глибини depth, де всі діти вузла — ОДНЕ й те саме піддерево.

    Логічно має arity**depth листків, але фізично лише depth+1 вузлів —
    саме тут мемоізація height дає експоненційний виграш.
    """
    node = TreeNode(0)
    for level in range(1, depth + 1):
        node = TreeNode(level, (node,) * arity)
    return node
```

4. У `algorithms.py` виконала базове завдання методички: чистий рекурсивний `quicksort` з випадковим pivot (повертає новий список), незмінний вузол `Node`

бінарного дерева і рекурсивні генератори `preorder`, `inorder`, `postorder`:

```python
def quicksort(xs: list[int]) -> list[int]:
    """Рекурсивний quicksort із випадковим pivot; повертає новий список."""
    if len(xs) <= 1:
        return xs[:]
    p = choice(xs)
    left = [x for x in xs if x < p]
    mid = [x for x in xs if x == p]
    right = [x for x in xs if x > p]
    return quicksort(left) + mid + quicksort(right)


@dataclass(frozen=True, slots=True)
class Node:
    """Вузол бінарного дерева."""

    key: int
    left: Node | None = None
    right: Node | None = None


def preorder(t: Node | None) -> Iterator[int]:
    """Корінь → ліве → праве (рекурсивний генератор)."""
    if t is None:
        return
    yield t.key
    yield from preorder(t.left)
    yield from preorder(t.right)


def inorder(t: Node | None) -> Iterator[int]:
    """Ліве → корінь → праве."""
    if t is None:
        return
    yield from inorder(t.left)
    yield t.key
    yield from inorder(t.right)


def postorder(t: Node | None) -> Iterator[int]:
    """Ліве → праве → корінь."""
    if t is None:
        return
    yield from postorder(t.left)
    yield from postorder(t.right)
    yield t.key

```

5. Додала ітеративні `bfs_level_order` (черга `deque`) і `dfs_preorder_iter` (стек)

для бінарного дерева та рекурсивну висоту:

```python
def bfs_level_order(t: Node | None) -> list[int]:
    """Ітеративний обхід у ширину через deque — не залежить від глибини стеку."""
    if t is None:
        return []
    q, out = deque([t]), []
    while q:
        n = q.popleft()
        out.append(n.key)
        if n.left:
            q.append(n.left)
        if n.right:
            q.append(n.right)
    return out


def dfs_preorder_iter(t: Node | None) -> list[int]:
    """Ітеративний preorder із власним стеком (правий кладемо першим)."""
    if t is None:
        return []
    stack, out = [t], []
    while stack:
        n = stack.pop()
        out.append(n.key)
        if n.right:
            stack.append(n.right)
        if n.left:
            stack.append(n.left)
    return out


def height(t: Node | None) -> int:
    """Висота дерева рекурсивно (порожнє дерево → 0)."""
    return 0 if t is None else 1 + max(height(t.left), height(t.right))

```

6. Реалізувала `fib` з `@lru_cache`, наївний `fib_naive` для порівняння та ітеративний `fib_iter` з O(1) пам'яті:

```python
@lru_cache(maxsize=100_000)
def fib(n: int) -> int:
    """Мемоізований Фібоначчі: O(n) замість експоненти."""
    if n < 0:
        raise ValueError("n must be >= 0")
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)


def fib_naive(n: int) -> int:
    """Той самий Фібоначчі без кешу — для порівняння часу."""
    return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)


def fib_iter(n: int) -> int:
    """Ітеративний Фібоначчі: O(n) часу, O(1) пам'яті, без стеку."""
    if n < 0:
        raise ValueError("n must be >= 0")
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

7. Демонстраційний `main.py` будує дерево з дев'яти вузлів, друкує обходи, результат пошуку, кількість вузлів і висоту, вимірює час `height` на

`shared_tree(22)` з кешем і без, показує `cache_info()`, час `fib` та ліміт рекурсії:

```python
def main() -> None:
    """Друкує результати та таблицю часу (лише I/O, обчислення чисті)."""
    tree = TreeNode(
        1,
        (
            TreeNode(2, (TreeNode(5), TreeNode(6, (TreeNode(9),)))),
            TreeNode(3),
            TreeNode(4, (TreeNode(7), TreeNode(8))),
        ),
    )
    print("preorder :", list(preorder(tree)), "dfs_iter:", dfs_iter(tree))
    print("postorder:", list(postorder(tree)))
    print("bfs_iter :", bfs_iter(tree))
    print("find 9   :", find_path(tree, 9), " find 42:", find_path(tree, 42))
    print("nodes:", count_nodes(tree), "height:", height(tree), height_iter(tree))

    big = shared_tree(22)

    def cached_run() -> int:
        height.cache_clear()
        return height(big)

    t_cached = timeit.timeit(cached_run, number=1)
    t_naive = timeit.timeit(lambda: height_naive(big), number=1)
    print(f"height(shared_tree(22)) cached: {t_cached * 1e3:.3f} ms")
    print(f"height(shared_tree(22)) naive : {t_naive * 1e3:.1f} ms")
    print("cache_info:", height.cache_info())

    fib.cache_clear()
    print("fib(30) cached:", f"{timeit.timeit(lambda: fib(30), number=1) * 1e3:.3f} ms")
    print(
        "fib(30) naive :",
        f"{timeit.timeit(lambda: fib_naive(30), number=1) * 1e3:.1f} ms",
    )
    print("fib(90) ==", fib(90), "iter:", fib_iter(90))

    print("quicksort:", quicksort([3, 1, 4, 1, 5, 9, 2, 6]))
    t = Node(4, Node(2, Node(1), Node(3)), Node(6, None, Node(7)))
    print("inorder:", list(inorder(t)), "bfs:", bfs_level_order(t))
    print("recursion limit:", sys.getrecursionlimit())


if __name__ == "__main__":
    main()
```

8. Написала тести у `tests/test_ntree.py`: послідовності рекурсивних обходів, збіг ітеративних версій з рекурсивними, граничний випадок одного вузла,

параметризований `find_path` (корінь, глибокий листок, відсутнє значення), висота та кількість вузлів:



```python
def sample_tree() -> TreeNode:
    #        1
    #      / | \
    #     2  3  4
    #    / \   / \
    #   5   6 7   8
    #       |
    #       9
    return TreeNode(
        1,
        (
            TreeNode(2, (TreeNode(5), TreeNode(6, (TreeNode(9),)))),
            TreeNode(3),
            TreeNode(4, (TreeNode(7), TreeNode(8))),
        ),
    )


def test_recursive_traversals() -> None:
    t = sample_tree()
    assert list(preorder(t)) == [1, 2, 5, 6, 9, 3, 4, 7, 8]
    assert list(postorder(t)) == [5, 9, 6, 2, 3, 7, 8, 4, 1]


def test_iterative_matches_recursive() -> None:
    t = sample_tree()
    assert dfs_iter(t) == list(preorder(t))
    assert bfs_iter(t) == [1, 2, 3, 4, 5, 6, 7, 8, 9]


def test_single_node_edges() -> None:
    leaf = TreeNode(7)
    assert list(preorder(leaf)) == list(postorder(leaf)) == [7]
    assert dfs_iter(leaf) == bfs_iter(leaf) == [7]
    assert height(leaf) == height_naive(leaf) == height_iter(leaf) == 1
    assert count_nodes(leaf) == 1
    assert find_path(leaf, 7) == (7,)
    assert find_path(leaf, 8) is None


@pytest.mark.parametrize(
    ("target", "path"),
    [(1, (1,)), (9, (1, 2, 6, 9)), (8, (1, 4, 8)), (3, (1, 3)), (42, None)],
)
def test_find_path(target: int, path: tuple[int, ...] | None) -> None:
    assert find_path(sample_tree(), target) == path
    assert contains(sample_tree(), target) is (path is not None)


def test_height_and_count() -> None:
    t = sample_tree()
    assert height(t) == height_naive(t) == height_iter(t) == 4
    assert count_nodes(t) == 9

```

9. Додала тести API кешу (`cache_info`/`cache_clear`: 23 стани і 22 попадання на спільному дереві), перевірку, що кешована висота збігається з наївною і

швидша за неї, тест глибокого лінійного дерева з 5000 вузлів (ітеративні функції працюють, рекурсивний `preorder` дає `RecursionError`) і тест незмінності дерева:

```python
def test_height_cache_api() -> None:
    height.cache_clear()
    t = shared_tree(10)
    assert height(t) == 11
    info = height.cache_info()
    assert info.currsize == 11 and info.hits == 10
    height(t)
    assert height.cache_info().hits == info.hits + 1
    height.cache_clear()
    assert height.cache_info().currsize == 0


def test_cached_matches_naive_and_faster() -> None:
    t = shared_tree(18)
    height.cache_clear()
    assert height(t) == height_naive(t) == 19
    t_naive = timeit.timeit(lambda: height_naive(t), number=1)

    def cached_run() -> int:
        height.cache_clear()
        return height(t)

    assert timeit.timeit(cached_run, number=1) < t_naive


def test_iterative_safe_on_deep_tree() -> None:
    # лінійне дерево глибини 5000: рекурсія впала б, ітерація — ні
    t = TreeNode(0)
    for k in range(1, 5001):
        t = TreeNode(k, (t,))
    assert height_iter(t) == 5001
    assert bfs_iter(t)[:3] == [5000, 4999, 4998]
    assert dfs_iter(t) == bfs_iter(t)
    with pytest.raises(RecursionError):
        list(preorder(t))


def test_tree_not_mutated() -> None:
    t = sample_tree()
    before = list(preorder(t))
    find_path(t, 9)
    height(t)
    bfs_iter(t)
    assert list(preorder(t)) == before
    with pytest.raises(AttributeError):
        t.value = 100  # type: ignore[misc]

```

10. Для property-тестів на Hypothesis описала стратегію випадкових дерев через `st.recursive`: ітеративні обходи збігаються з рекурсивними, довжина обходу дорівнює кількості вузлів, три версії висоти рівні, а шлях `find_path` до будь-

якого значення починається з кореня, закінчується шуканим значенням і не довший за висоту:

```python
trees = st.recursive(
    st.builds(TreeNode, st.integers(0, 50)),
    lambda kids: st.builds(TreeNode, st.integers(0, 50), st.tuples(kids, kids, kids)),
    max_leaves=30,
)


@given(trees)
def test_property_iterative_equals_recursive(t: TreeNode) -> None:
    assert dfs_iter(t) == list(preorder(t))
    assert sorted(bfs_iter(t)) == sorted(postorder(t))
    assert len(dfs_iter(t)) == count_nodes(t)
    assert height(t) == height_naive(t) == height_iter(t)


@given(trees)
def test_property_find_path_is_valid_path(t: TreeNode) -> None:
    for value in bfs_iter(t):
        path = find_path(t, value)
        assert path is not None and path[0] == t.value and path[-1] == value
        assert len(path) <= height(t)
```

11. Тести базового завдання з методички: quicksort проти `sorted` (включно з property-тестом), послідовності обходів бінарного дерева і глибоке лінійне дерево, Фібоначчі з перевіркою кешу та відхиленням від'ємних аргументів:

```python
def test_fib_small() -> None:
    expected = [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert [fib(i) for i in range(10)] == expected
    assert [fib_naive(i) for i in range(10)] == expected


def test_fib_iter_matches() -> None:
    for n in range(50):
        assert fib(n) == fib_iter(n)


def test_cache_clear_and_info() -> None:
    fib.cache_clear()
    assert fib(0) == 0
    fib(25)
    info = fib.cache_info()
    assert info.currsize == 26
    assert info.hits > 0


def test_negative_rejected() -> None:
    with pytest.raises(ValueError):
        fib(-1)
    with pytest.raises(ValueError):
        fib_iter(-1)
```

```python
def sample_tree() -> Node:
    #      4
    #     / \
    #    2   6
    #   / \   \
    #  1   3   7
    return Node(4, left=Node(2, Node(1), Node(3)), right=Node(6, None, Node(7)))


def test_traversals() -> None:
    t = sample_tree()
    assert list(preorder(t)) == [4, 2, 1, 3, 6, 7]
    assert list(inorder(t)) == [1, 2, 3, 4, 6, 7]
    assert list(postorder(t)) == [1, 3, 2, 7, 6, 4]
    assert bfs_level_order(t) == [4, 2, 6, 1, 3, 7]
    assert dfs_preorder_iter(t) == [4, 2, 1, 3, 6, 7]
```

12. Запустила демонстрацію: мемоізована `height` на дереві глибини 22 зі спільними піддеревами виконується за 0.06 мс проти 1.8 с без кешу (23 стани, 22 попадання), `fib(30)` — 0.01 мс проти 98 мс; ліміт рекурсії — 1000:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab04> python main.py
preorder : [1, 2, 5, 6, 9, 3, 4, 7, 8] dfs_iter: [1, 2, 5, 6, 9, 3, 4, 7, 8]
postorder: [5, 9, 6, 2, 3, 7, 8, 4, 1]
bfs_iter : [1, 2, 3, 4, 5, 6, 7, 8, 9]
find 9   : (1, 2, 6, 9)  find 42: None
nodes: 9 height: 4 4
height(shared_tree(22)) cached: 0.055 ms
height(shared_tree(22)) naive : 1849.6 ms
cache_info: CacheInfo(hits=22, misses=23, maxsize=100000, currsize=23)
fib(30) cached: 0.012 ms
fib(30) naive : 102.6 ms
fib(90) == 2880067194370816120 iter: 2880067194370816120
quicksort: [1, 1, 2, 3, 4, 5, 6, 9]
inorder: [1, 2, 3, 4, 6, 7] bfs: [4, 2, 6, 1, 3, 7]
recursion limit: 1000
```

13. Перевірила проєкт автоматичними інструментами: усі 26 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab04> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 26 items

tests\test_fib.py::test_fib_small PASSED                                 [  3%]
tests\test_fib.py::test_fib_iter_matches PASSED                          [  7%]
tests\test_fib.py::test_cache_clear_and_info PASSED                      [ 11%]
tests\test_fib.py::test_negative_rejected PASSED                         [ 15%]
tests\test_ntree.py::test_recursive_traversals PASSED                    [ 19%]
tests\test_ntree.py::test_iterative_matches_recursive PASSED             [ 23%]
tests\test_ntree.py::test_single_node_edges PASSED                       [ 26%]
tests\test_ntree.py::test_find_path[1-path0] PASSED                      [ 30%]
tests\test_ntree.py::test_find_path[9-path1] PASSED                      [ 34%]
tests\test_ntree.py::test_find_path[8-path2] PASSED                      [ 38%]
tests\test_ntree.py::test_find_path[3-path3] PASSED                      [ 42%]
tests\test_ntree.py::test_find_path[42-None] PASSED                      [ 46%]
tests\test_ntree.py::test_height_and_count PASSED                        [ 50%]
tests\test_ntree.py::test_height_cache_api PASSED                        [ 53%]
tests\test_ntree.py::test_cached_matches_naive_and_faster PASSED         [ 57%]
tests\test_ntree.py::test_iterative_safe_on_deep_tree PASSED             [ 61%]
tests\test_ntree.py::test_tree_not_mutated PASSED                        [ 65%]
tests\test_ntree.py::test_property_iterative_equals_recursive PASSED     [ 69%]
tests\test_ntree.py::test_property_find_path_is_valid_path PASSED        [ 73%]
tests\test_quicksort.py::test_quicksort_basic PASSED                     [ 76%]
tests\test_quicksort.py::test_quicksort_random PASSED                    [ 80%]
tests\test_quicksort.py::test_quicksort_does_not_mutate PASSED           [ 84%]
tests\test_quicksort.py::test_property_matches_sorted PASSED             [ 88%]
tests\test_tree.py::test_traversals PASSED                               [ 92%]
tests\test_tree.py::test_empty_and_height PASSED                         [ 96%]
tests\test_tree.py::test_iterative_matches_recursive_on_deep_tree PASSED [100%]

============================== 26 passed in 0.91s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab04> python -m mypy .
Success: no issues found in 8 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab04> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab04> python -m black --check .
All done! ✨ 🍰 ✨
8 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізувала рекурсивні алгоритми для n-арного дерева з `tuple` дітей (варіант 14): обходи preorder і postorder, пошук значення зі шляхом від кореня та висоту дерева, а також quicksort, обходи бінарного дерева і Фібоначчі з методички. Для кожної рекурсії чітко сформульовано базовий випадок (листок, `n < 2`) і крок. Мемоізація через `@lru_cache` на незмінних вузлах перетворила експоненційний підрахунок висоти дерева зі спільними піддеревами на лінійний, що підтверджено вимірюванням часу і `cache_info()`. Ітеративні DFS, BFS і висота по рівнях безпечніші за рекурсію на глибоких деревах, бо CPython обмежує стек 1000 кадрами і не оптимізує хвостові виклики, що показав тест на лінійному дереві з 5000 вузлів. Усі функції чисті й не змінюють дерево, що підтверджено юніт- і property-тестами, mypy, ruff і black.